



# 21. Genetic Algorithms: The Knapsack Problem

How many times have I revisited the Knapsack Problem?

[Previous Section:](#)

 [20. Genetic Algorithms](#)

[Contents:](#)

[1. One-Max Problem with Python](#)

[2. Fitness Functions in the Knapsack Problem](#)

[3. Knapsack Problem: The Genetic Approach](#)

[4. Knapsack Problem: Step-by-Step Genetic Evolution](#)

[Evaluate](#)

[Summary](#)

[References](#)

This is the continuation of the previous Genetic Algorithm post, and if you've made it this far, you already understand something important:

┆ *Genetic Algorithms are not about writing perfect solutions, they are about evolving them.*

Instead of solving a problem directly, we let a population of possible solutions compete, mutate, and improve over time. The weak ones disappear. The strong ones survive. And generation after generation, the answers get better.

But now comes the real question: **What kind of problems truly *benefit* from this idea?**

Because Genetic Algorithms are not just clever, they are *practical*. Especially when the search space is too large, too complex, or too chaotic for traditional methods.

In this section, we step into two classic problems—problems that look simple on the surface, but quickly become impossible if approached naively.

- The first is the **One-Max Problem**. A deceptively simple challenge that helps us understand how Genetic Algorithms behave in code. No tricks. No hidden complexity. Just pure evolution at work.

- And the second is the **Knapsack Problem**. A problem that sounds almost trivial:

*"You have a bag with limited capacity... and a set of items, each with weight and value. What should you take?"*

But the moment you try to solve it optimally, you realize something. Every decision affects every other decision:

- Take one item → you lose space for another
- Skip one item → you might lose a better combination

And suddenly, it's not a simple choice anymore. It's a combinatorial explosion. This is where Genetic Algorithms shine. They don't try every possibility. They don't rely on greedy shortcuts. Instead, they explore. They adapt. They evolve toward better solutions.

So in this section, we won't just talk about Genetic Algorithms. We will *watch them think*.

## 1. One-Max Problem with Python

I recommend you revisit the previous **Genetic Algorithms** section if you want a deeper understanding of the One-Max problem. Here, we shift our focus to something more practical: **implementation**. Because at the end of the day, understanding an algorithm is one thing. But seeing it *run*, evolve, and improve solutions in real time? That's where everything clicks.

The goal of this section is simple: We will build a complete Genetic Algorithm from scratch to solve the One-Max problem.

But there's something more important happening behind the scenes. This implementation is not just for One-Max. It is a **template**. A reusable structure that we will later adapt to solve a much more complex problem → The **Knapsack Problem**.

So as you go through the code, don't just read it line by line. Try to see the pattern:

Initialization → Evaluation → Selection → Crossover → Mutation → Replacement

This is the *DNA* of every Genetic Algorithm.

Let's begin. We start with the most fundamental building block: **creating a population**.

- The `random_genome(string_length)` function generates a random binary chromosome of a given length. Each gene is either `0` or `1`, chosen randomly. This forms the initial population—the starting point of evolution.

```
# Genetic Algorithms: One-Max Problem
# Single-Point Crossover
import random

def random_genome(string_length):
    # Generate a random string with a given length
    return [random.choice([0, 1]) for _ in range(string_length)]
```

- Next, we need a way to evaluate how “good” each solution is. The `fitness(genome)` function computes the fitness value for each chromosome.
  - For the One-Max problem, fitness is simply the number of 1s in the string.
  - More 1s → better solution.

```
def fitness(genome):
    # Find fitness: The number of 1s in the string
    fitness_list = []
    for i in range(len(genome)):
        genome_one = [genome[i][j] for j in range(len(genome[i]))
                       if genome[i][j] == 1]
        fitness = len(genome_one)
        fitness_list.append(fitness)
    return fitness_list
```

- Now comes **selection**. The `choose_parent(genome, top_parents)` function selects the strongest individuals based on fitness. Only the best survive... and get to reproduce.

```
def choose_parent(genome, top_parents):
    fitness_list = fitness(genome)
    merged_list = [(genome[i], fitness_list[i]) for i in range(len(genome))]
    sorted_list = sorted(merged_list, key=lambda x: x[1], reverse=True)
    return sorted_list[:top_parents]
```

- With parents selected, we move to **crossover**. This is where new solutions are created by combining existing ones. The `crossover(genome, top_parents)` function performs **single-point crossover**:
  - Pick a random split point
  - Swap genetic material
  - Produce offspring

```
def crossover(genome, top_parents):
    parent1, parent2 = choose_parent(genome, top_parents)
    parent1, parent2 = parent1[0], parent2[0]

    cross_point = random.randint(0, len(genome))

    offspring1 = parent1[:cross_point] + parent2[cross_point:]
    offspring2 = parent2[:cross_point] + parent1[cross_point:]
```

```
return offspring1, offspring2
```

- But crossover alone is not enough. Without randomness, evolution stagnates. That's where **mutation** comes in. The `mutation()` function randomly flips bits with a small probability.
  - This introduces variation and prevents the algorithm from getting stuck.

```
def mutation(genome, top_parents, mutation_rate):
    offspring1, offspring2 = crossover(genome, top_parents)

    offspring1_flip = [bit if random.random() > mutation_rate else 1 -
bit
                        for bit in offspring1]
    offspring2_flip = [bit if random.random() > mutation_rate else 1 -
bit
                        for bit in offspring2]

    return offspring1_flip, offspring2_flip
```

- Now we combine everything into **evolution**. The `new_generation()` function creates the next population:
  - Evaluate fitness
  - Generate offspring
  - Replace weaker individuals

```
def new_generation(genome, top_parents, mutation_rate):
    fitness_list = fitness(genome)
    offspring1, offspring2 = list(mutation(genome, top_parents, mutatio
n_rate))

    merged_list = [(genome[i], fitness_list[i]) for i in range(len(geno
me))]
    sorted_list = sorted(merged_list, key=lambda x: x[1], reverse=True)

    sorted_genome = [gen[0] for gen in sorted_list]

    top_genome = [offspring1] + [offspring2]
    sorted_genome = top_genome + [
        geno for geno in sorted_genome if geno not in sorted_genome[:to
p_parents]
    ]
```

```
return sorted_genome
```

We also need a stopping condition. The `is_solution_found()` function checks whether we have reached the optimal solution—all 1s.

```
def is_solution_found(genome, string_length):
    for g in genome:
        if sum(g) == string_length:
            return True, g
    return False, []
```

Finally, the **engine** of the algorithm. The `main()` function runs everything:

```
def main():
    print("One Max Problem")

    string_length = 5
    population_size = 4
    mutation_rate = 1 / string_length
    generations = 100
    top_parents = 2

    genome = [random_genome(string_length) for _ in range(population_size)]

    for gen in range(generations):
        print(f"\nGeneration {gen + 1}:")
        for i, individual in enumerate(genome):
            print(f"- {i+1}: {individual} | Fitness: {sum(individual)}")

        found, solution = is_solution_found(genome, string_length)

        if found:
            print(f"\nSolution found in generation {gen + 1}: {solution}")
            break

        genome = new_generation(genome, top_parents, mutation_rate)

    else:
        print("\nSolution not found within the generation limit.")
```

```
if __name__ == "__main__":  
    main()
```

At first glance, this problem might feel *too simple*. Just maximize the number of 1s? But that's exactly the point.

This problem strips away all complexity, and lets you see the **pure mechanics** of a Genetic Algorithm. And once you understand this. You're ready for something much more interesting.

Because in the next section. We take this exact structure—and use it to solve the **Knapsack Problem**.

Let's run the code (you can try it in any Python environment to see the full evolution process). But instead of getting lost in all the intermediate generations, let's focus on the moment that actually matters:

```
Solution found in generation 16: [1, 1, 1, 1, 1]
```

At first glance, this looks simple. But there's something deeper happening here. The algorithm didn't *compute* this answer directly.

It didn't follow a fixed path → It didn't "know" the solution beforehand → **It discovered it.**

Now here's the interesting part: **Generation 16** is not special.

Run the program again, and you might see:

- **Generation 35**
- **Generation 46**
- Sometimes, it might even take the full **100 generations**

And occasionally? It might converge *very quickly*. This is not a bug. This is not inefficiency. → This is the **nature of Genetic Algorithms**.

Because Genetic Algorithms are not deterministic. They rely on: Random initialization; Random crossover points; and Random mutations → Every run is a *different evolutionary story*.

- Some populations get lucky early → strong genes appear quickly
- Some struggle → weak combinations take longer to improve

But over time, the same pattern emerges: The population *moves toward better solutions*

This is what makes Genetic Algorithms powerful: They don't guarantee the fastest path. They don't guarantee the same path. But they are very good at **finding good solutions in complex spaces**.

And this is the key mindset shift:

- Traditional algorithms ask → *"What is the exact sequence of steps to solve this?"*
- Genetic Algorithms ask → *"What if we let solutions compete... and evolve?"*

So when you see: `Solution found in generation 16`

Don't just read it as output. Read it as: A population started randomly, struggled, combined, mutated, improved. And eventually, **evolved into the answer**.

And now that you've seen this in its purest form. We're ready to take the next step. Because in the **Knapsack Problem**. There is no obvious "all 1s" solution anymore. Only trade-offs. Only constraints. Only *strategy*.

## 2. Fitness Functions in the Knapsack Problem

*(I deliberately waited until this section to introduce fitness functions properly. Because here, they finally matter.)*

**Fitness Functions: Constraints for Optimization:** In many real-world optimization problems—like **knapsack, scheduling, or routing**—the goal is not just to maximize something (value, profit, efficiency...).

There's always a catch. A **constraint**: Weight limits? Time windows? Capacity restrictions? And suddenly, the problem is no longer just "find the best solution". It becomes: *"Find the best solution that is also valid."*

Here's where Genetic Algorithms feel a bit unusual. Unlike classical optimization methods, GAs don't naturally "respect" constraints. They don't stop and say: *"This solution is invalid, discard it."* → Instead, they do something smarter (and a bit more subtle): They **embed constraints into the fitness function itself**.

### Penalty-Based Constraints (Penalty Function)

This is the most common approach. If a solution violates constraints. We don't remove it immediately. We simply make it **less attractive** → We *penalize* it.

**Example: Knapsack Problem - Let:**

- Total weight =  $W_{total}$ ;
- Maximum capacity =  $W_{max}$ ;
- Total value =  $V_{total}$

Then the fitness function becomes:

→ If the solution is valid:  $Fitness = V_{total}$

→ If the solution violates the constraint: Fitness is reduced

We can express this idea formally:

```
if W_total <= W_max:
    fitness = V_total
else:
    fitness = V_total - penalty_factor * (W_total - W_max)
```

**What does this actually do?**

- If a solution slightly exceeds the weight → small penalty

- If it exceeds a lot → heavy penalty

So over time:

- Bad (infeasible) solutions → become weaker
- Good (feasible) solutions → dominate the population

And just like that, invalid solutions **naturally disappear through evolution**. No hard rules. No strict filtering. Just survival pressure.

### Repair-Based Constraints

There's another approach—and it's more *interventionist*. Instead of punishing bad solutions, we **fix them**. After crossover or mutation:

- If a solution breaks constraints → we repair it
- Only valid solutions are allowed to continue

### Common repair strategies:

- Remove items until weight is valid
- Drop the least valuable items
- Adjust values or structure

This approach is especially useful in structured problems:

- **Traveling Salesman Problem (TSP)** → must visit each city exactly once
- **Knapsack Problem** → remove items until weight fits
- **Scheduling Problems** → resolve time conflicts

### Two Philosophies (and both are valid)

At this point, you can feel the difference:

- **Penalty-based** → Let evolution handle it
- **Repair-based** → Enforce correctness manually

There's no universal "best" choice. It depends on the problem.

### Genetic Algorithms: Explained Using Biological Logic

Now let's step back for a moment, and connect this to intuition. Because everything we're doing here is inspired by something very familiar: → **Biological reproduction**

In nature:

- A mother and a father contribute genetic material
- Their child inherits a mix of both
- Small mutations introduce variation

Genetic Algorithms follow the same idea.



### Mapping the analogy:

- **Parents** → Candidate solutions
- **Genes** → Individual values (bits, numbers, items)
- **Crossover** → Mixing solutions
- **Mutation** → Small random changes

### Single-Point Crossover (Intuition)

- Choose a random split point
- Take the first part from Parent A
- Take the rest from Parent B

```
Parent A: 11001 | 0110
Parent B: 00111 | 1001

Child   : 11001 | 1001
```

A clean splice. Simple, but powerful.

### Two-Point Crossover (More flexible)

- Choose two points
- Swap the middle segment

Example:

```
Parent A: 11 | 001 | 0110
Parent B: 00 | 111 | 1001

Child   : 11 | 111 | 0110
```

Now instead of a single cut, we exchange an entire block of "genetic material".

Genetic Algorithms don't *construct* solutions step by step.

They **breed** them: Combine good traits → Mutate slightly → Let the best survive

And over time, solutions evolve.

But here's the key insight for the Knapsack Problem: The algorithm doesn't "understand" weight limits. It doesn't "know" what is valid. All it sees is: Fitness.

So if you design the fitness function well. You are not just evaluating solutions. You are **guiding evolution itself**. And that is where Genetic Algorithms become truly powerful.

### 3. Knapsack Problem: The Genetic Approach

Now, we finally reach the **core** of this section.

Up until now, everything we've built—the One-Max problem, the crossover logic, mutation, selection—was preparing us for this moment. Because the **Knapsack Problem** is where Genetic Algorithms start to feel *real*.

**What is the Knapsack Problem?** The Knapsack Problem is a classic optimization problem.

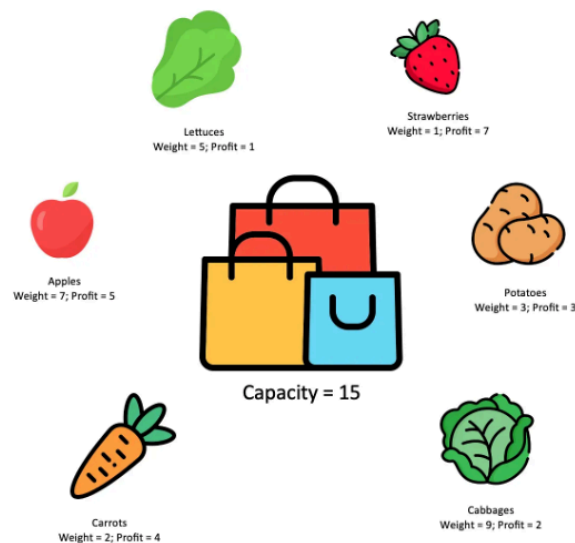
The idea is simple: You have a container (a *knapsack*) with **limited capacity**, and a set of items.

Each item has a weight  $w$  and a value  $v$  (or profit)

Your goal? Select a subset of items such that: The **total weight** does not exceed the capacity and the **total value** is maximized

**A Simple Example** - Imagine you're carrying a grocery bag with a maximum capacity of **15 units**. You are given the following items (reference the image below):

```
items = [[7, 5], [2, 4], [1, 7], [9, 2], [5, 1], [3, 3]]
```



Each pair represents: `[weight, value]` → So:

- Item 1 → weight = 7, value = 5
- Item 2 → weight = 2, value = 4
- Item 3 → weight = 1, value = 7
- ... and so on

**The Challenge** - At first, it looks straightforward. Just pick the items with the highest value, right? Not quite.

Because every decision comes with a trade-off:

- A high-value item might be too heavy

- Many small items might outperform one big item
- The “best” combination is not always obvious

And here’s the problem: The number of possible combinations grows **exponentially**. For  $n$  items, there are  $2^n$  possible subsets. Even for a small list, that becomes overwhelming.

**Why Genetic Algorithms?** Traditional methods (like brute force or dynamic programming) can solve this problem. But as the number of items increases:

- Brute force → becomes too slow
- Exact methods → become computationally expensive

This is where Genetic Algorithms step in.

**The Genetic Perspective** - Instead of checking every possible combination. We **evolve** good solutions.

**Encoding the Problem** - Each solution is represented as a **chromosome**. For example:

```
[1, 0, 1, 0, 1, 0]
```

This means:

**Take item 1** → **Skip item 2** → **Take item 3** → **Skip item 4** → **Take item 5** → **Skip item 6**

**Evolution in Action** - Now everything connects back to what we learned:

- **Population** → Different item selections
- **Fitness** → Total value (with penalties if overweight)
- **Selection** → Choose the best combinations
- **Crossover** → Mix selections between solutions
- **Mutation** → Randomly flip decisions (add/remove items)

Generation after generation. Bad combinations disappear. Good combinations survive. Better combinations emerge.

The algorithm doesn’t try to “solve” the problem directly. It doesn’t search systematically. Instead. It *explores* the solution space through evolution.

**Why This Matters** - In the One-Max problem, the goal was obvious → All 1s

But in the Knapsack Problem? There is **no obvious perfect pattern**. Only trade-offs, constraints, and competing objectives.

This is what makes the Knapsack Problem so powerful as a learning tool. It forces you to think differently: Not in terms of exact answers. But in terms of **better and better approximations**. And with Genetic Algorithms: You don’t need to find the perfect solution immediately. You just need to **evolve toward it**.

And now we are ready for the most exciting part: Turning this idea into code.

## 4. Knapsack Problem: Step-by-Step Genetic Evolution

We move from theory into **execution**.

This is where everything you've learned—fitness, selection, crossover—comes together into a **working evolutionary system**.

### Step 0: Initialize the Parameters

We begin by defining the core elements of both the problem and the algorithm:

```
items = [[7, 5], [2, 4], [1, 7], [9, 2], [5, 1], [3, 3]]
capacity = 15
generations = 100
population_size = 10
mutation_rate = 0.05
```

Each parameter plays a very specific role:

- `items` → Each item is `[weight, value]`
- `capacity` → Maximum weight the knapsack can carry
- `generations` → Number of evolutionary cycles
- `population_size` → Number of candidate solutions per generation
- `mutation_rate` → Probability of random gene changes

**Important Note** - This is a **0/1 Knapsack Problem**:

- `1` → Item is selected
- `0` → Item is not selected
- No partial selection allowed

So each solution becomes a binary string: `[1, 0, 1, 0, 0, 1]`

Which means: → Take items 1, 3, and 6

### Problem Setup

| Item | Weight | Value |
|------|--------|-------|
| 1    | 7      | 5     |
| 2    | 2      | 4     |
| 3    | 1      | 7     |
| 4    | 9      | 2     |
| 5    | 5      | 1     |
| 6    | 3      | 3     |

Capacity = **15**

### Step 1: Initialize the Random Population

We start with **random guesses**. Each genome is a binary string of length 6 (since there are 6 items).

```
import random

def initial_population(items):
    return [random.choice([0, 1]) for _ in range(len(items))]
```

We generate **10 genomes** → this is our initial population.

### Example Population

| Genome | Binary Representation | Meaning (Selected Items) |
|--------|-----------------------|--------------------------|
| 1      | 1 0 1 0 0 1           | Items 1, 3, 6            |
| 2      | 0 1 0 0 1 1           | Items 2, 5, 6            |
| 3      | 1 1 1 0 0 0           | Items 1, 2, 3            |
| 4      | 0 0 1 0 0 0           | Item 3                   |
| 5      | 1 0 1 1 0 0           | Items 1, 3, 4            |
| 6      | 1 1 0 0 0 0           | Items 1, 2               |
| 7      | 0 0 0 1 1 0           | Items 4, 5               |
| 8      | 1 0 0 0 1 0           | Items 1, 5               |
| 9      | 0 1 1 0 0 1           | Items 2, 3, 6            |
| 10     | 1 0 0 1 0 1           | Items 1, 4, 6            |

At this stage, these are just random combinations. No intelligence yet. No optimization. Just raw material for evolution.

### Step 2: Evaluate Fitness

Now we ask the key question: *How good is each solution?*

For each genome: Compute total weight → Compute total value → Check constraint (capacity = 15)

| Genome | Binary Representation | Meaning (Selected Items) | Total Weight     | Total Value      | Valid? |
|--------|-----------------------|--------------------------|------------------|------------------|--------|
| 1      | 1 0 1 0 0 1           | Items 1, 3, 6            | $7 + 1 + 3 = 11$ | $5 + 7 + 3 = 15$ | Yes    |
| 2      | 0 1 0 0 1 1           | Items 2, 5, 6            | $2 + 5 + 3 = 10$ | $4 + 1 + 3 = 10$ | Yes    |
| 3      | 1 1 1 0 0 0           | Items 1, 2, 3            | $7 + 2 + 1 = 10$ | $5 + 4 + 7 = 16$ | Yes    |
| 4      | 0 0 1 0 0 0           | Item 3                   | 1                | 7                | Yes    |
| 5      | 1 0 1 1 0 0           | Items 1, 3, 4            | $7 + 1 + 9 = 17$ | $5 + 7 + 2 = 14$ | No     |
| 6      | 1 1 0 0 0 0           | Items 1, 2               | $7 + 2 = 9$      | $5 + 4 = 9$      | Yes    |
| 7      | 0 0 0 1 1 0           | Items 4, 5               | $9 + 5 = 14$     | $2 + 1 = 3$      | Yes    |

|    |             |               |                  |                  |     |
|----|-------------|---------------|------------------|------------------|-----|
| 8  | 1 0 0 0 1 0 | Items 1, 5    | $7 + 5 = 12$     | $5 + 1 = 6$      | Yes |
| 9  | 0 1 1 0 0 1 | Items 2, 3, 6 | $2 + 1 + 3 = 6$  | $4 + 7 + 3 = 14$ | Yes |
| 10 | 1 0 0 1 0 1 | Items 1, 4, 6 | $7 + 9 + 3 = 19$ | $5 + 2 + 3 = 10$ | No  |

Invalid solutions exceed capacity → removed or penalized.

### Python Implementation:

```
def fitness(genomes):
    fitness_list = []

    genomes_weight = [[items[j][0] if genomes[i][j] == 1 else 0
                       for j in range(len(genomes[i]))]
                      for i in range(len(genomes))]

    genomes_profit = [[items[j][1] if genomes[i][j] == 1 else 0
                      for j in range(len(genomes[i]))]
                      for i in range(len(genomes))]

    for gen in range(len(genomes)):
        total_weight = sum(genomes_weight[gen])
        total_profit = sum(genomes_profit[gen])

        if capacity >= total_weight > 0:
            fitness_list.append((genomes[gen], total_profit))

    return fitness_list
```

This is where your earlier concept comes alive: The **fitness function controls evolution**

- Valid + high value → survive
- Invalid → disappear

No explicit “rules”, just **selection pressure**.

### Step 3: Reproduction and Selection

Now comes survival of the fittest. We select the **best genomes** to act as parents.

#### Strategy: Elitism

- Sort by fitness (profit)
- Keep only the top performers
- In this case → top **10%** (minimum 2 parents)

#### Example Parents

| Parent | Genome      | Value | Items       |
|--------|-------------|-------|-------------|
| 1      | 1 1 1 0 0 0 | 16    | Items 1,2,3 |
| 2      | 1 0 1 0 0 1 | 15    | Items 1,3,6 |

## Python Implementation

```
def reproduction(genomes):
    fitness_list = fitness(genomes)
    sorted_list = sorted(fitness_list, key=lambda x: x[1], reverse=True)
    top_parents = max(2, len(sorted_list) // 10)

    return sorted_list[:top_parents]
```

We just simulated natural selection:

- Weak solutions → eliminated
- Strong solutions → reproduce

At this point: We have a population. We evaluated fitness. We selected the best parents

Now comes the most exciting part: **Crossover and Mutation**. This is where:

- New solutions are created
- Good traits combine
- Randomness introduces new possibilities

And most importantly. This is where evolution actually *happens*. Because up until now? We were just choosing. Next, we start **creating**.

## Step 4: Crossover and Mutation

We arrive at the **heart of the Genetic Algorithm**. Up until now, we've only been selecting the best solutions. But selection alone does not create anything new.

This step answers the real question: *How do new (and better) solutions emerge?*

### Crossover: Creating New Solutions

Crossover is the process of **combining two parent solutions** to create offspring. Given two strong parents:

Given two strong parents:

```
Parent 1: 1 1 1 0 0 0
Parent 2: 1 0 1 0 0 1
```

### Single-Point Crossover (Intuition)

Assume we choose a crossover point after the first two bits:

- Offspring 1 → first part from Parent 1, rest from Parent 2
- Offspring 2 → first part from Parent 2, rest from Parent 1

```
Offspring 1: 1 1 | 1 0 0 1 → [1, 1, 1, 0, 0, 1]
Offspring 2: 1 0 | 1 0 0 0 → [1, 0, 1, 0, 0, 0]
```

### Evaluate the Offspring

**Offspring 1:** [1, 1, 1, 0, 0, 1]

- Items: 1, 2, 3, 6
- Weight:  $7 + 2 + 1 + 3 = 13$
- Profit:  $5 + 4 + 7 + 3 = 19$  → Valid solution

**Offspring 2:** [1, 0, 1, 0, 0, 0]

- Items: 1, 3
- Weight:  $7 + 1 = 8$
- Profit:  $5 + 7 = 12$  → Valid solution

Offspring 1 (profit = 19) is **better than both parents**. This is the power of Genetic Algorithms: They don't just preserve good solutions. They can **create better ones through recombination**.

**Uniform Crossover (Actual Implementation):** While single-point crossover is easy to visualize, the actual algorithm uses **uniform crossover**.

Instead of splitting once: Each gene is chosen independently → 50% chance from Parent 1 and 50% chance from Parent 2. **Example:**

```
Parent 1: 1 1 1 0 0 0
Parent 2: 1 0 1 0 0 1

Random choice: [P1, P2, P1, P2, P1, P2]
```

Possible offspring:

```
Offspring 1: 1 0 1 0 0 1
Offspring 2: 1 1 1 0 0 0
```

### Evaluate

- [1, 0, 1, 0, 0, 1] → Profit = 15 → Valid solution
- [1, 1, 1, 0, 0, 0] → Profit = 16 → Valid solution

Good, but not yet better than **19**.



**Mutation: Introducing Randomness** - Crossover recombines existing genes. But what if the current population is stuck? That's where mutation comes in.

### Mutation Mechanism

- Each bit has a small chance (5%) to flip, either `1 → 0` or `0 → 1`

### Example

```
Before mutation: [1, 1, 1, 0, 0, 1]
After mutation  : [1, 1, 1, 0, 0, 0]
```

### Re-evaluation

`[1, 1, 1, 0, 0, 0]` → Items 1, 2, 3 → Weight = 10 → Profit = 16

**Why Mutation Matters** - Mutation does not always improve solutions immediately. But it is essential because it maintains diversity, prevents premature convergence, and helps escape local optima → Without mutation... evolution can stagnate.

**Handling Invalid Solutions** - Now comes a critical question: *What if crossover or mutation produces an invalid solution?* We don't explicitly discard it immediately. We let the **fitness function handle it**.

### Validation Rule

```
if capacity >= total_weight > 0:
    fitness_list.append((genomes[gen], total_profit))
```

- Valid solution → kept
- Invalid (overweight) → ignored

So naturally: Invalid solutions **do not survive** to the next generation

### Python Implementation (Crossover + Mutation Combined)

```
def crossover(genomes):
    parents = reproduction(genomes)
    parents = [parent[0] for parent in parents]

    offspring = []

    while len(offspring) < int(population_size * 0.9):
        parent1, parent2 = random.sample(parents, 2)

        child = [parent1[i] if random.random() < 0.5 else parent2[i]
                  for i in range(len(parent1))]

        mutated_child = mutation(child, mutation_rate)

        offspring.append(mutated_child)
```

```
return offspring
```

```
def mutation(genomes, mutation_rate):  
    return [bit if random.random() > mutation_rate else 1 - bit  
            for bit in genomes]
```

**Main Execution Loop** - This ties everything together:

```
def main():  
    print("Knapsack Problem: Genetic Algorithm")  
  
    global generations  
    genomes = [initial_population(items) for _ in range(population_size)]  
  
    best_solution = None  
    best_profit = 0  
    avg_fitness_history = []  
  
    for generation in range(generations):  
        fitness_list = fitness(genomes)  
  
        if fitness_list:  
            top_solution = max(fitness_list, key=lambda x: x[1])  
  
            avg_fitness = sum(fit[1] for fit in fitness_list) / len(fitness_list)  
            avg_fitness_history.append(avg_fitness)  
  
            if top_solution[1] > best_profit:  
                best_solution, best_profit = top_solution  
  
            print(f"Generation {generation + 1}: Best Profit = {top_solution[1]}, Solution = {top_solution[0]}")  
            print(f"Average Fitness {avg_fitness}")  
  
        genomes = crossover(genomes)  
  
    print(f"\nBest Overall Solution: {best_solution}\nProfit: {best_profit}")
```

```
main()
```

This step is where everything becomes *alive*:

- Selection chooses the best
- Crossover creates new combinations
- Mutation introduces randomness
- Fitness filters reality

And together. They simulate evolution. And over generations. The algorithm doesn't just search. It **adapts, learns, and evolves toward optimal solutions**.

### Final Result: What Did the Algorithm Actually Find?

After running the Genetic Algorithm for **100 generations**, we now focus on what truly matters: The **best solution discovered during the entire evolutionary process**

```
Best Overall Solution: [1, 1, 1, 0, 0, 1]
Profit: 19
```

### Interpreting the Solution

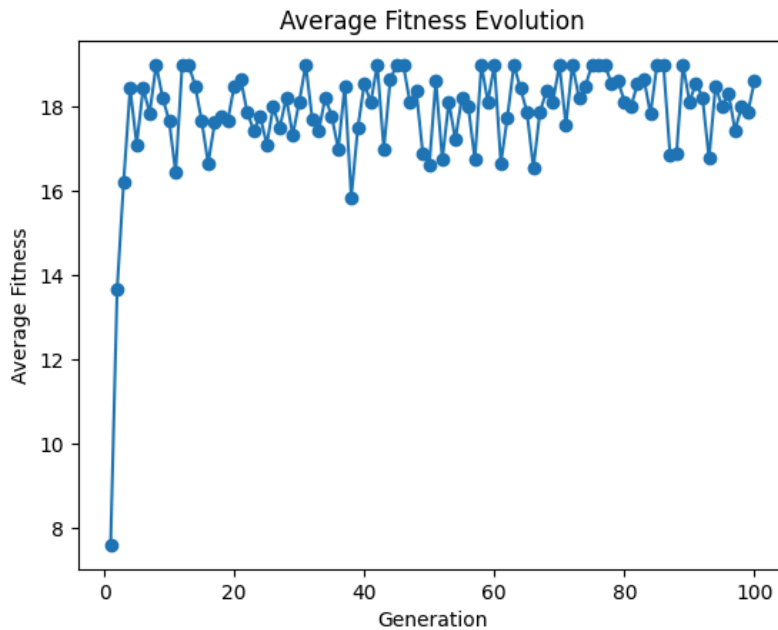
- This binary string tells a complete story: [1, 1, 1, 0, 0, 1]

Meaning: Item 1 → Item 2 → Item 3 → Item 4 → Item 5 → Item 6 (the green color represents the selected item)

- **Selected Items**
  - Items: **1, 2, 3, 6**
  - Total weight:  $7 + 2 + 1 + 3 = 13$  → within capacity
  - Total profit:  $5 + 4 + 7 + 3 = 19$  → the best weight
- This solution respects the **capacity constraint ( $\leq 15$ )** and achieves the **highest profit found during evolution**. And most importantly:
  - It was **not explicitly constructed**
  - It was **discovered through evolution**

### About the Fitness Plot

When you run the code, you'll also see a graph of **average fitness over generations** (*The following should be the plot that is returned if you run the code correctly*):



What does it typically show?

- Early generations ( 0 → ~10–15 ): rapid improvement
- Later generations: plateau around 16–19

This tells us something important about Genetic Algorithms:

- At the beginning → lots of random exploration
- Quickly → good solutions emerge
- Eventually → improvement slows down

This is called **convergence**.

**But why does it stagnate?** Because:

- The population becomes similar
- Less diversity → fewer new ideas
- Mutation becomes the only source of variation

### The Role of Randomness

Here's the key idea you should never forget: Genetic Algorithms are **stochastic**. This means: Different initial populations. Different crossover points. Different mutations. **Different results every run**

### What You Might Observe

Run the code multiple times, and you may see:

- Same best solution (profit = 19)
- Slight variations in generations needed
- Occasionally, different valid solutions

### Recommendation: Run the algorithm at least 10 times

Why? Because a single run does **not guarantee** the global optimum and only gives you **one evolutionary outcome**

Multiple runs increase reliability, help you compare solutions, and give a better sense of consistency.

*You can download the full code here (file type `.py`)*

[`GA\_knapsack\_problem.py`](#)

## Summary

Genetic Algorithms are powerful, but they are not magic. They require a well-designed **fitness function**, proper handling of **constraints**, and careful tuning of parameters

And most importantly:

They don't solve problems directly → They **evolve solutions over time**

So when you see: `Best Overall Solution: [1, 1, 1, 0, 0, 1]`

What you're really seeing is: A population that started randomly, explored countless combinations, adapted through selection, crossover, and mutation. And eventually **evolved into a high-quality solution under constraints**. And that is the essence of Genetic Algorithms.

## References

<https://arpitbhayani.me/blogs/genetic-knapsack/>

<https://www.kaggle.com/code/moazeldsokyx/genetic-algorithm-for-knapsack-problem-solving>

## Next Section:

 [22. Recommender Systems with Collaborative Filtering](#)